```
int product(int num1, int num2) {
    return num1 * num2;
}

float product(float num1, float num2) {
    return num1 * num2;
}

double product(double num1, double num2) {
    return num1 * num2;
}

int main() {

    cout << product(2, 3) << endl;
    cout << product(1.1f, 2.2f) << endl;
    cout << product(1.1, 2.2) << endl;

    return 0;
}
```

Output

6
2.42
2.42

- Notice duplication of code in all three variants of product function
- Is there a way to consolidate these different functions? As what they are doing remains same but only thing that differs is the type they handle

```
template<typename T>
T product(T num1, T num2) {
    return num1 * num2;
}

int main() {

    cout << product(2, 3) << endl;
    cout << product(1.1f, 2.2f) << endl;
    cout << product(1.1, 2.2) << endl;

    return 0;
}
```

Output

```
6
2.42
2.42
```

- Compiler generates actual code for function ***product*** (for types with which it has been called in the program). Here compiler will instantiate three overloads of function ***product*** as we are calling it with int, float and double types.

```
template<typename T>
T product(T num1, T num2) {
    return num1 * num2;
}
int main() {
    cout << product(2, 3) << endl;
    cout << product(1.1f, 2.2f) << endl;

    // cout << product(3, 4.5) << endl;
    // error: no matching function for call to ‘product(int, double)
    // note: deduced conflicting types for parameter ‘T’ (‘int’ and ‘double’)

    cout << product<int>(3, 4.5) << endl;
    cout << product<float>(3, 4.5) << endl;

    // string s1 = "Hello ", s2 = "world!\n";
    // error: no match for ‘operator*’
    // cout << product(s1, s2) << endl;
}
```

Output

```
6
2.42
12
13.5
```

- If template arguments are not explicitly specified during function call, then they are deduced by compiler based on function arguments and function template parameters.
  - Compiler will raise an error if there is conflict in deduction
- We may call function with explicit template arguments
  - e.g. product<int>(3, 4.5)
    - It will now consider T as int
- Compiler will instantiate only two overloads of function *product* as we are calling it with int and float types.

```
template<typename T>
T product(T num1, T num2) {
    return num1 * num2;
}
```

Is same as (atleast for our basic use)

```
template<class T>
T product(T num1, T num2) {
    return num1 * num2;
}
```

- Even when you use class keyword you can call this function with fundamental data types as T
- So, prefer use of keyowrd ***typename*** over keyword ***class*** as it is close to its actual behaviour

```
template<typename T>
void myswap(T &num1, T &num2) {
    T temp;
    temp = num1;
    num1 = num2;
    num2 = temp;
}
int main() {
    int int1 = 5, int2 = 7;
    cout << "Before swap: int1 = " << int1;
    cout << " and int2 = " << int2 << endl;
    myswap(int1, int2);
    cout << "After swap: int1 = " << int1;
    cout << " and int2 = " << int2 << endl;

    double double1 = 5, double2 = 7;
    cout << "Before swap: double1 = " << double1;
    cout << " and double2 = " << double2 << endl;
    myswap(double1, double2);
    cout << "After swap: double1 = " << double1;
    cout << " and double2 = " << double2 << endl;

    return 0;
}
```

Output

```
Before swap: int1 = 5 and int2 = 7
After swap: int1 = 7 and int2 = 5
Before swap: double1 = 5 and double2 = 7
After swap: double1 = 7 and double2 = 5
```

- Just another example, Templates can have references or pointers of T as well
- T is generally used as typename, but it could be any valid identifier

```
template<typename T>
T product(T num1, T num2) {
    cout << "Template function called\n";
    return num1 * num2;
}
float product(float num1, float num2) {
    cout << "Non-template function called\n";
    return num1 * num2;
}
int product(int num1, int num2, int num3) {
    cout << "Non-template function with three argumets
called\n";
    return num1 * num2 * num3;
}
int main() {
    cout << product(2, 3) << endl;
    cout << product(1.1f, 2.2f) << endl;
    cout << product<float>(1.1f, 2.2f) << endl;
    cout << product<>(1.1f, 2.2f) << endl;
    cout << product(1.1, 2.2) << endl;
    // Not an exact match
    cout << product(1.1, 2.2, 3.3) << endl;
    return 0;}
```

Output

```
Template function called
6
Non-template function called
2.42
Template function called
2.42
Template function called
2.42
Template function called
2.42
Non-template function with three argumets called
6
```

- Function template and non-template function can have same name
- Overload resolution for function template and non-template function is as follows (when called without explicit template arguments)
  - Call non-template function that has exact match
  - Instantiate function from function template (if possible with exact match)
  - Try overload resolution with non-template overloads of a function

```
template<typename T>
T product(T num1, int num2, T num3) {
    cout << "Template function called\n";
    return num1 * num2 * num3;
}

int main() {



    cout << product(2.5, 3, 2.5) << endl;

    cout << product(2.5, 3.5, 2.5) << endl;

    // error: no matching function for call to ‘product(double, double, int)’
    // note: deduced conflicting types for parameter ‘T’ (‘double’ and ‘int’)
    // cout << product(2.5, 3.5, 2) << endl;

    return 0;
}
```

Output

```
Template function called
18.75
Template function called
18.75
```

- Function arguments can be mixed
- Implicit conversion can take place for fixed type arguments of a function

```
template<typename T1, typename T2>
T2 product(T2 num1, T2 num2, T1 num3) {
    cout << "Template function called\n";
    return num1 * num2 * num3;
}

int main() {

    // error: no matching function for call to ‘product(double, int, double)’
    // note: deduced conflicting types for parameter ‘T2’ (‘double’ and ‘int’)
    // cout << product(2.5, 3, 2.5) << endl;

    cout << product(2.5, 3.5, 2.5) << endl;

    cout << product(2.5, 3.5, 2) << endl;

    return 0;
}
```

Output

Template function called
21.875
Template function called
17.5

- Function templates can have multiple template parameters (e.g. T1 and T2 in this case)
- There is no correlation between sequence of template parameters and sequence of function parameters.

```
template<typename T1, typename T2>
T1 product(T2 num1, T2 num2, T1 num3) {
    cout << "Template function called\n";
    T1 result;
    result = num1 * num2 * num3;
    return result;
}

int main() {

    cout << product<double,int>(2.5, 3, 2.5) << endl;

    cout << product(2.5, 3.5, 2.5) << endl;

    cout << product(2.5, 3.5, 2) << endl;

    return 0;
}
```

Output

```
Template function called
15
Template function called
21.875
Template function called
17
```

- Type specified in template parameters can also be used in function body
- Template arguments can be specified during function call even when template has multiple parameters